# Course Project: Part 2

Qubit Allocation & Routing: Beating (or Matching) SABRE

**At a glance**

**Weight:** ∼12% *[exact allocation released with parameter sheet]*
**Team:** Same groups as Part 1 (3–4 students)
**Language:** Python – your router drives your Part 1 simulator as the backend
**Spec released:** Nov 1 — **Deadline:** Nov 17
**Baseline:** Qiskit's SABRE transpiler (`qiskit 2.x`, provided on the lab image)

## 1 Overview

A quantum circuit is written assuming that any qubit can interact with any other. Real devices are not like that: a superconducting chip only supports two-qubit gates between *physically adjacent* qubits, described by a **coupling graph**. Making an arbitrary circuit executable therefore has two steps: (1) an **initial mapping** of logical qubits onto physical ones, and (2) **routing** — inserting SWAP gates to bring interacting qubits together as the circuit proceeds. Both are done by the compiler, and how well they are done decides how many extra gates (hence how much error) a NISQ circuit accumulates.

Part 2 asks your group to build this compiler stage, run its output through the Part 1 simulator to *prove it is correct*, and benchmark it against the industry-standard **SABRE** heuristic as implemented in Qiskit.

**Why this stress-tests your Part 1 simulator**

Routing *produces* circuits. A near-all-to-all circuit (e.g. QFT) on a sparse line explodes in depth and two-qubit count once SWAPs are inserted. Your correctness check is to run the *original* and the *routed* circuit on your Part 1 statevector backend and show they are the same computation (after undoing the qubit relabelling). The routed circuits are deeper and SWAP-heavy, exercising your simulator far harder than the input.

**What Part 2 is about.** The intellectual core is not new physics — it is an *algorithms and systems* problem: implement SABRE correctly, make its output provably equivalent to the input, measure it honestly against a strong baseline, and then *improve* on it. There is no QUBO, no variational optimization, and no new C++: everything runs in Python on top of the simulator you already built.

# 2 The Problem, Precisely

## 2.1 Inputs

- A **logical circuit**: an ordered list of gates on $n$ logical qubits (single-qubit gates, and two-qubit gates `CNOT`, `CZ`, `CU`, `SWAP`).
- A **coupling graph** $G = (V, E)$: physical qubits $V$ and the pairs $E$ that admit a native two-qubit gate.

## 2.2 Output

A **routed circuit** on physical qubits in which *every* two-qubit gate acts on a pair $(u, v) \in E$, together with the initial and final logical→physical layouts.

## 2.3 Correctness

The routed circuit must be **functionally equivalent** to the original. Because routing permutes which physical qubit carries which logical value, the routed output is the original output *up to the final layout permutation*; equivalence is checked after undoing that permutation.

## 2.4 Objective

Among all correct routings, minimise the overhead — primarily the **number of inserted SWAPs** (each SWAP = 3 CNOTs), and secondarily the added circuit **depth**.

> Minimum-SWAP routing is **NP-hard** (Siraichi et al., CGO 2018): the initial-mapping sub-problem contains subgraph-isomorphism, and routing is a token-swapping problem on $G$. This is why production compilers use heuristics such as SABRE rather than exact solvers.

# 3 Provided Harness

Three files are provided in `part2/` (do not modify the first and third):

| File | Role |
|---|---|
| `routing_common.py` | **(provided)** `Op`, `RoutingResult`, a `RecordingSimulator` that captures any Part-1 `circuits.py` generator into an `Op` list, coupling-graph builders, a trusted numpy statevector oracle, and the legality / fidelity / metric helpers. |
| `reference_router.py` | **(starter)** the Core-tier router (identity layout + shortest-path SWAP insertion). Correct but not clever — your starting point and a sanity reference. |
| `grader/route_harness.py` | **(provided)** the auto-grader (see §7). Run it yourself, early and often. |

## 3.1 The `Router` contract

Your submission is `part2/router.py` defining a class `Router` with exactly this interface:

```
from routing_common import Op, RoutingResult, neighbours, shortest_path

class Router:
    name = "sabre"                          # short label used in reports

    def __init__(self, edges):              # edges: list of (u, v),
        undirected
        ...

    def route(self, ops, n):
        # ops : list[Op] on n LOGICAL qubits
        # n   : number of logical qubits
        # return RoutingResult(ops=<physical Op list>,
        #                      initial=<logical->physical layout, len n
            >,
        #                      final=<logical->physical layout after
            routing>)
        ...
```

`RoutingResult.ops` must contain only legal two-qubit gates, and must differ from the input only by (a) relabelling qubits per `initial` and (b) inserted `SWAP` gates. An `Op` is `Op(gate, qubits, params)`; capture an input circuit with, e.g., `ops = capture(circuits.qft, 8)`.

## 4 Coupling Graphs and Benchmark Circuits

**Coupling graphs** (builders in `routing_common`): a **line**, a **2D grid**, and the IBM 27-qubit **heavy-hex** map. Routing difficulty increases from left to right (diameter grows, degree stays low).

**Circuit families** (Part-1 generators, captured via `capture`): GHZ, QFT (routing-heavy, near all-to-all), Bernstein–Vazirani, 1D-chain QAOA, a brick-wall `random_universal` circuit, and `near_clifford`. The ripple-carry adder from `ReversibleLogic.tex` is an optional structured case.

## 5 What to Build (Tiers)

**Core – correctness floor**

Reproduce and understand the reference router: an identity initial layout and **shortest-path SWAP insertion**. Make it pass the correctness gate (legality + fidelity $\geq 0.999$) on every benchmark circuit and graph.

**Main – implement SABRE**

Implement the SABRE heuristic properly:

- a **front layer** of currently-executable gates, with a look-ahead *extended set* and a distance-based SWAP cost with a decay factor;
- the **reverse-traversal** initial mapping (route forward, then backward, feeding the final layout back as the next initial layout, iterating).

Benchmark against Qiskit's SABRE across the full circuit $\times$ graph matrix.

**Stretch – improve on SABRE**

Make *one* concrete, justified improvement over your own baseline SABRE and demonstrate the gain with an ablation. Candidate directions:

- larger / weighted look-ahead window, or a tuned decay schedule;
- a better initial layout (more reverse-traversal passes, or a graph-similarity seed);
- **bridge gates** instead of a SWAP when a CNOT is exactly two hops apart;
- commutation-aware gate reordering before routing.

Beating *your own* baseline SABRE is the bar; beating *Qiskit* is a bonus. "We matched Qiskit on the grid but lost 15% on heavy-hex, because . . . " is a valid, well-graded result.

## 6 Metrics and the Qiskit Comparison

To keep the comparison fair (Qiskit also optimises and decomposes), the protocol is fixed and the grader applies it for you:

- **Inserted SWAPs** — the headline metric. Qiskit baseline: `transpile(qc, coupling_map, layout_method='sabre', routing_method='sabre', optimization_level=0)`, counting `swap` before decomposition.
- **Depth / two-qubit count** — reported at Qiskit `optimization_level=3` (its strongest setting), as the "industrial baseline" for added depth.
- Report **overhead ratios** relative to the unrouted circuit, so your numbers and Qiskit's lie on the same normalised axis.

## 7 Auto-grading and What Is Graded by Hand

Most of Part 2 is **machine-checked**. The grader (`route_harness.py`) runs, per group, in one invocation:

1. **Legality gate** — every routed two-qubit gate lies on a coupling edge.
2. **Equivalence gate** — routed $\equiv$ original at fidelity $\geq 0.999$, computed by a trusted numpy oracle that undoes the output layout permutation. This needs *neither* Qiskit *nor* your C++ backend, so it cannot be gamed.
3. **Metrics** — SWAPs / two-qubit count / depth, and the Qiskit SABRE baseline, emitted as a CSV.
4. **Improvement delta** — if you submit `router_improved.py`, its SWAP total vs. your baseline.

```
# self-test with the reference router
python route_harness.py
# grade a group
python route_harness.py --router-dir ../../group7/part2 --router router
   \
                        --improved router_improved --csv results/group7.
                           csv
```

The correctness gate is a hard prerequisite: **no metric credit is awarded for a routing that fails legality or equivalence**, exactly as in Part 1. A routed circuit that gives a different distribution is almost always a missing SWAP or a mishandled output permutation — the grader tells you which circuit fails.

Graded by hand, as elsewhere in this course: the short **report**, the **design journal**, and the **oral defense**. There is no written derivation to mark.

## 8 Deliverables

**Part 2 Deliverables – due Nov 17**

1. **Router** (`part2/router.py`) implementing SABRE behind the `Router` contract; passes the correctness gate on all benchmarks.
2. **Improvement** (`part2/router_improved.py`) — one justified change over your baseline, plus the ablation plot.
3. **Benchmark study** — results table (your baseline SABRE vs. Qiskit): SWAPs / two-qubit count / depth across the circuit × graph matrix, as overhead ratios.
4. **Equivalence note** — a short explanation of how you track and undo the output layout permutation, with the grader's fidelity evidence.
5. **Simulator stress plot** — Part-1 backend runtime vs. qubit count for unrouted vs. routed QFT (and/or random), tying Part 2 back to Part 1.
6. **Short report** (3–4 pages) — design decisions; honest discussion of where you match/beat/lose to Qiskit and why.
7. **Design-journal update** for the Nov 1–17 window.

## 9 Assessment

Mark allocation

Exact component weights released with the parameter sheet (Nov 1).

| Component | Basis |
|---|---|
| Correctness (legality + equivalence) | Auto-graded gate; hard prerequisite for any further credit. |
| SABRE implementation | Front layer, look-ahead cost, reverse-traversal layout; measured against Qiskit. |
| Improvement & ablation | A real gain over your own baseline; sound experimental method. |
| Benchmark study | Complete circuit × graph matrix; correct normalised comparison. |
| Report quality | Honesty about match/loss vs. Qiskit; clarity of design decisions. |
| Oral defense (per member) | Individual questioning – see below. |

### 9.1 Oral defense

Each member is individually questioned. Representative questions:

- Your routed circuit gives a different bit-string distribution from the original. Is it wrong, or is it the output layout permutation? Show me the mapping that reconciles them.
- Walk me through one SABRE look-ahead decision on this front layer. What does the decay factor do?
- Why is minimum-SWAP routing NP-hard? What problem does the initial-mapping step reduce from?
- Where does your router beat Qiskit, where does it lose, and why?
- When is a *bridge* gate cheaper than a SWAP?

A group that only used an LLM to emit code typically cannot explain why a missing SWAP breaks equivalence while a wrong *final layout* does not, or why SABRE's reverse traversal helps the initial mapping. The design journal and oral defense are where that understanding is assessed — not the absence of AI tools (which are permitted).

## 10 Reading List

1. Li, Ding & Xie, "Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices," *ASPLOS* 2019. **Required — this is SABRE.** Read the cost function, the extended set, and the reverse-traversal initial mapping in full.
2. Siraichi, Santos, Collange & Pereira, "Qubit Allocation," *CGO* 2018. **Required for the NP-hardness argument** and the exact-vs-heuristic framing.
3. Qiskit transpiler documentation: `SabreLayout` and `SabreSwap` passes. Use as the baseline you compare against.
4. (Optional) Cowtan et al., "On the Qubit Routing Problem" (t|ket⟩), *TQC* 2019 — an alternative heuristic worth contrasting.

**Reading priority**

Start with Paper 1 (SABRE); the whole Main tier is a faithful implementation of it. Paper 2 gives you the hardness framing you need for the oral defense.